

◆ CS EDUCATION ◆

🔍

CI / CD

26.08.07
배선아



교육 목표

1. CI와 CD의 개념 및 차이
2. CI/CD 파이프라인의 동작 과정
3. 환경별 배포와 배포 전략
4. 안전한 배포를 위한 고려 사항

금요일 오후 5시, 배포 요청

기획자

“로그인 수정사항
오늘 안에 반영해주세요”



개발자

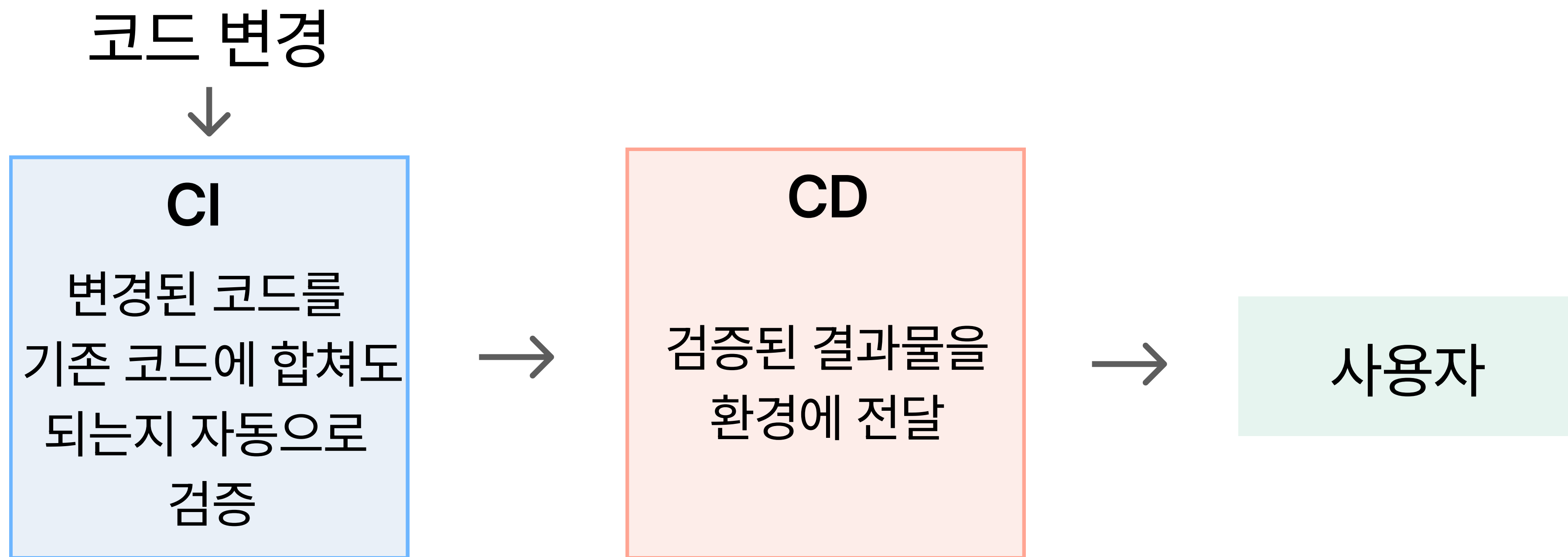
“잠시만요...”



코드 병합 → 테스트 → 빌드 → 서버 반영 → 최종 확인

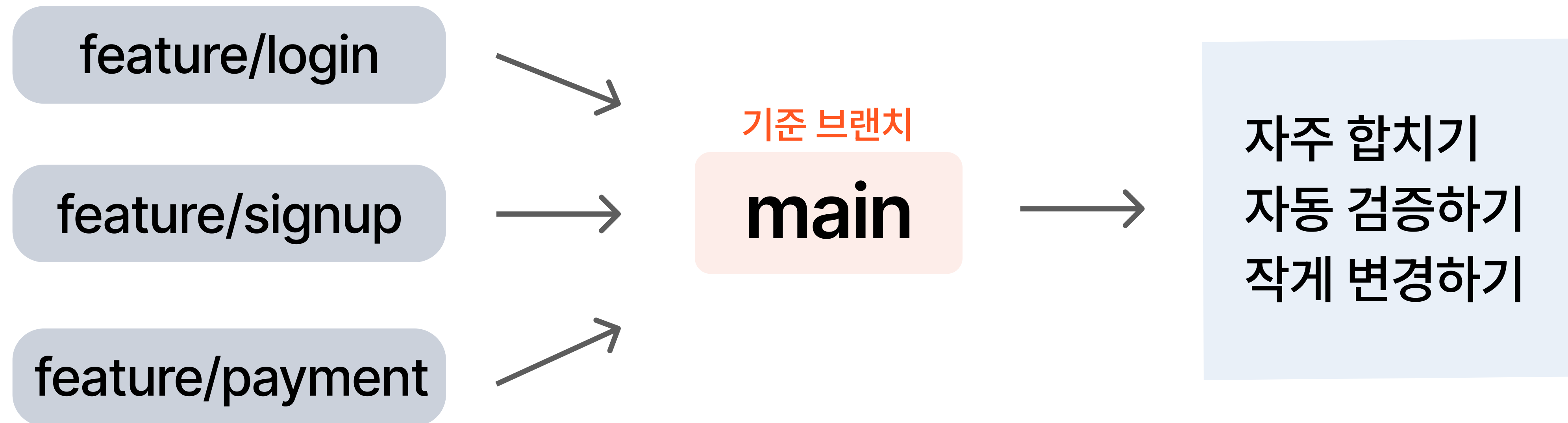
이 과정을 매번 사람이 직접 한다면?

반복되는 검사와 배포를 자동화한다면?



더 자주, 더 안전하게 결과물을 전달한다

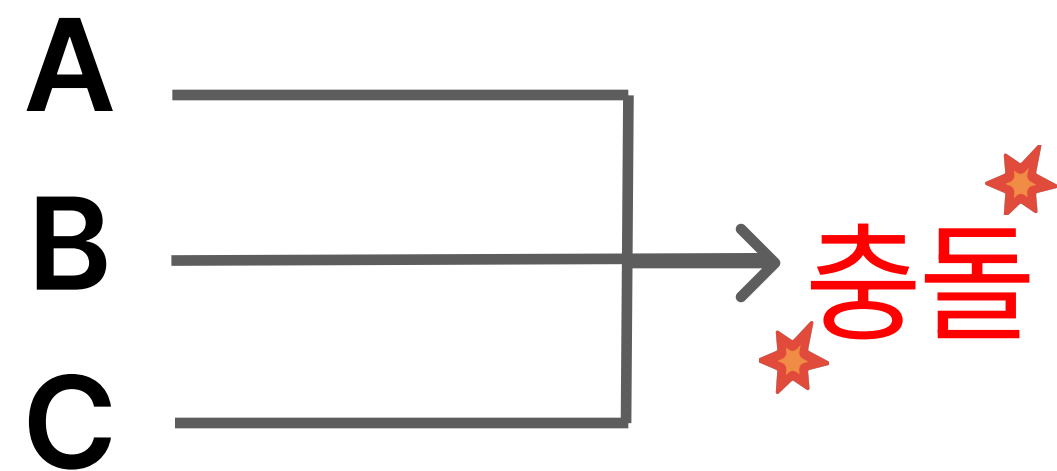
CI, Continuous Integration



작은 변경을 자주 통합하면 문제의 원인을 찾기 쉬워진다

왜 코드를 자주 합쳐야 할까?

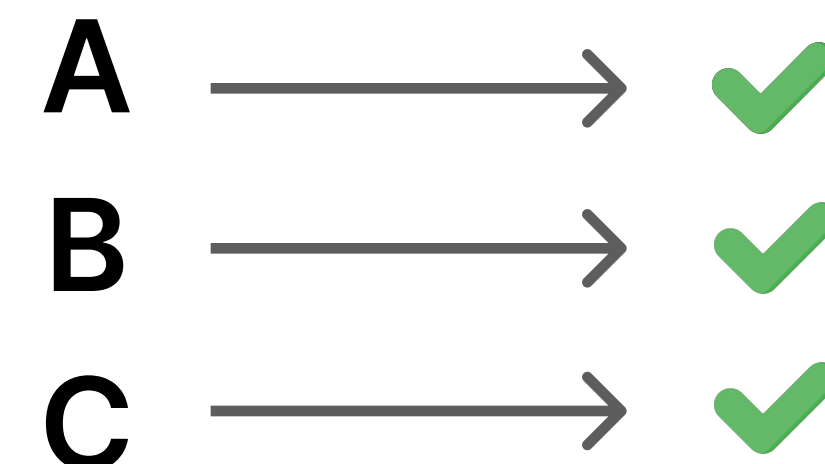
큰 변경, 늦은 통합



- 변경 20개
- 충돌 원인 불명확
- 수정 범위 큼

VS

작은 변경, 잦은 통합



- 변경 2~3개
- 최근 변경만 확인
- 수정 범위 작음

작은 변경은 검증하고 수정하기 쉽다

파이프라인 Trigger

Trigger: 파이프라인을 시작하는 이벤트

Pull Request

PR 생성

Push

브랜치에
코드 반영

Schedule

정해진 시간

Manual

사용자가
직접 실행

CI/CD

이벤트가 발생하면 미리 정의한 작업이 실행된다

Trigger에 따른 실행 범위

Pull Request



빠른 검사

병합 가능 확인

Main Push



Staging 배포

검증 환경 배포

Version Tag / Manual



Production 배포

(승인 필요 시) 실행

Version Tag: “이 버전을 배포하겠다”는 표시

모든 이벤트에서 모든 작업을 실행할 필요는 없다

파이프라인 내부 구조(Workflow, Job, Step)

Workflow (전체 자동화 흐름)

Job: Frontend

Step1 Checkout
Step2 Install
Step3 Test

Job: Backend

Step1 Checkout
Step2 Test
Step3 Build

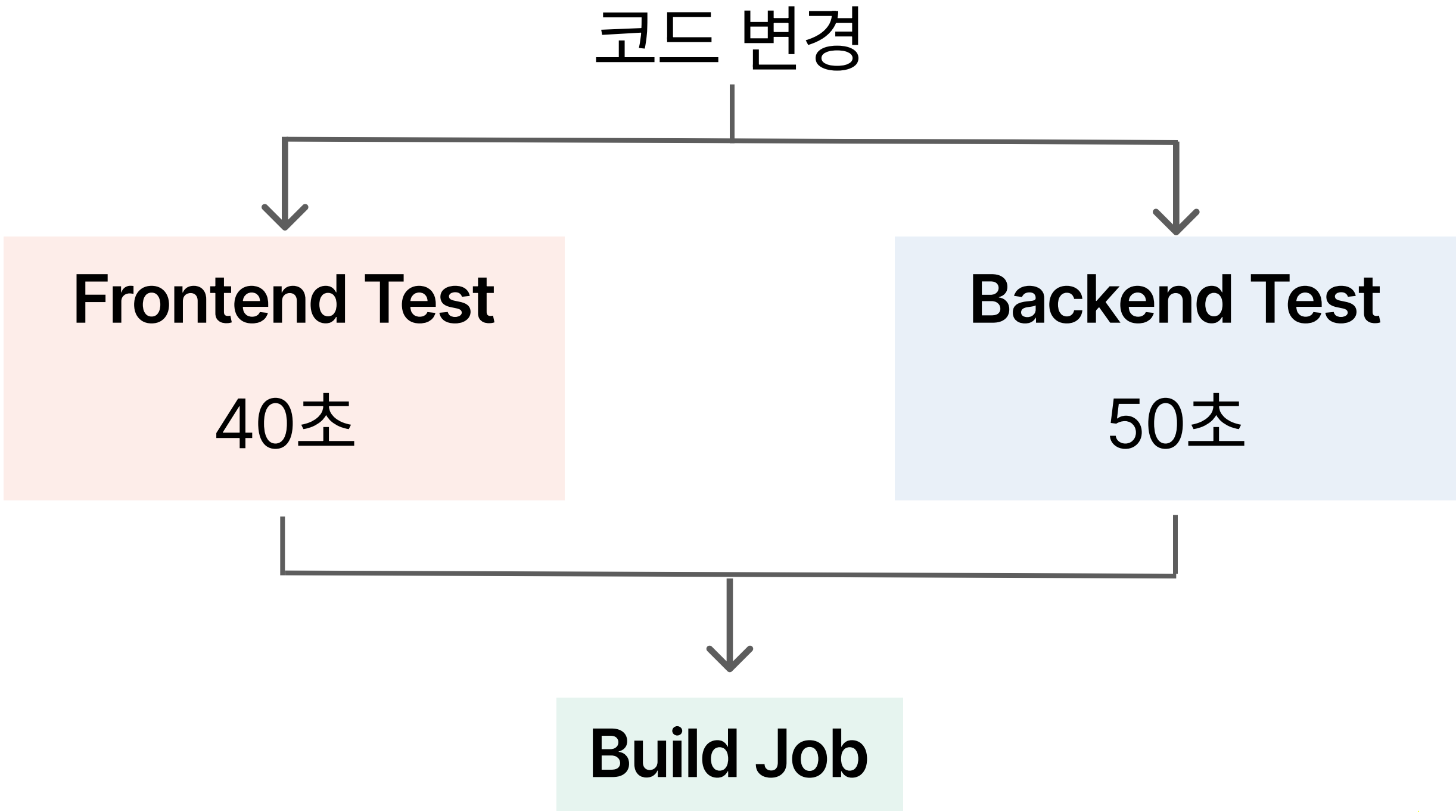
Workflow = 전체 흐름

Job = 독립적인 작업 묶음

Step = 실제 실행 단위

Workflow 안에 Job이 있고, Job안에 Step이 있다

Job의 병렬 실행



순차 실행 90초

★ 대기시간
확보 가능
병렬 실행 50초

CI의 검사 항목

Lint

코드 규칙 검사

Test

기능 동작 검증

Build

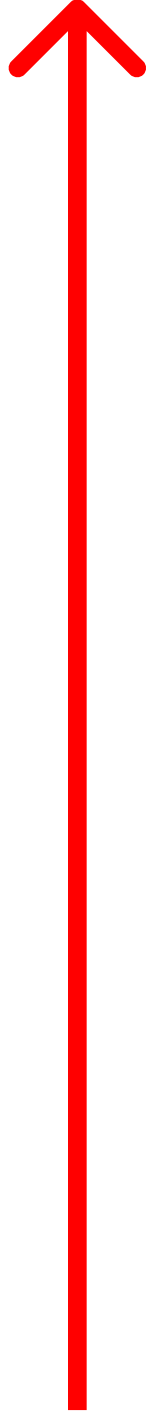
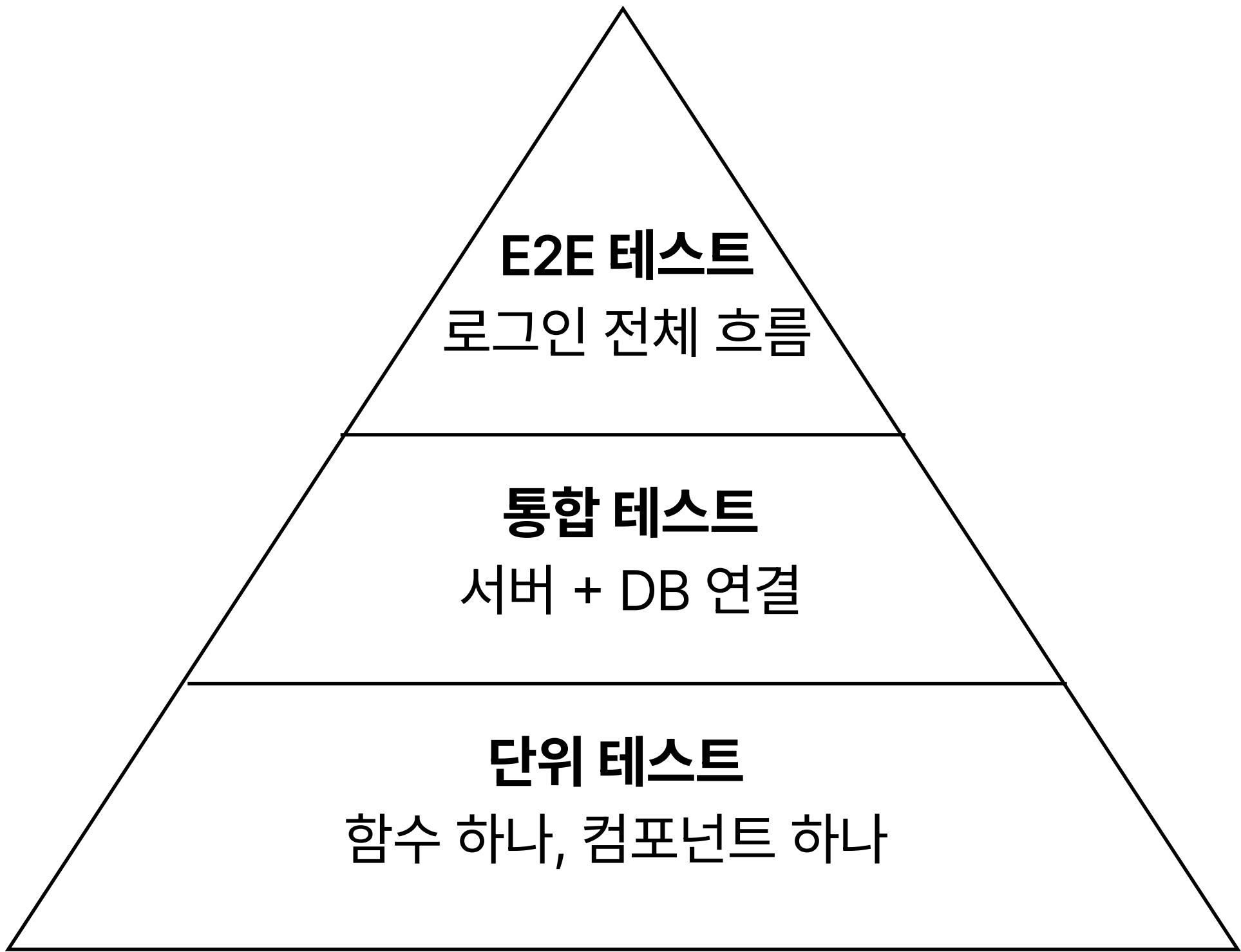
빌드 가능 여부 확인

Security

취약점·패키지 검사

CI는 모든 문제를 찾는 것이 아니라, 팀이 정한 기준을 반복 검사한다

테스트 피라미드



위로 갈수록

- 범위 큼
- 속도 느림
- 비용 큼

한 종류만 선택하는 것이 아니라 목적에 맞게 조합한다

CI 실패

❌ Build #127

- | | |
|-------------|-----|
| ✓ 코드 스타일 검사 | 12s |
| ✓ 단위 테스트 | 31s |
| ✗ 애플리케이션 빌드 | 8s |

Error: API_URL is not defined



- 사용자 영향 없음
- 원인 확인
- 수정 후 재실행

실패를 막는 것보다, 실패를 빠르게 발견하고 이해하는 것이 중요하다

Artifact

Artifact: 빌드를 통해 만들어진 배포 가능한 결과물

Frontend
dist 폴더

Backend
JAR 파일

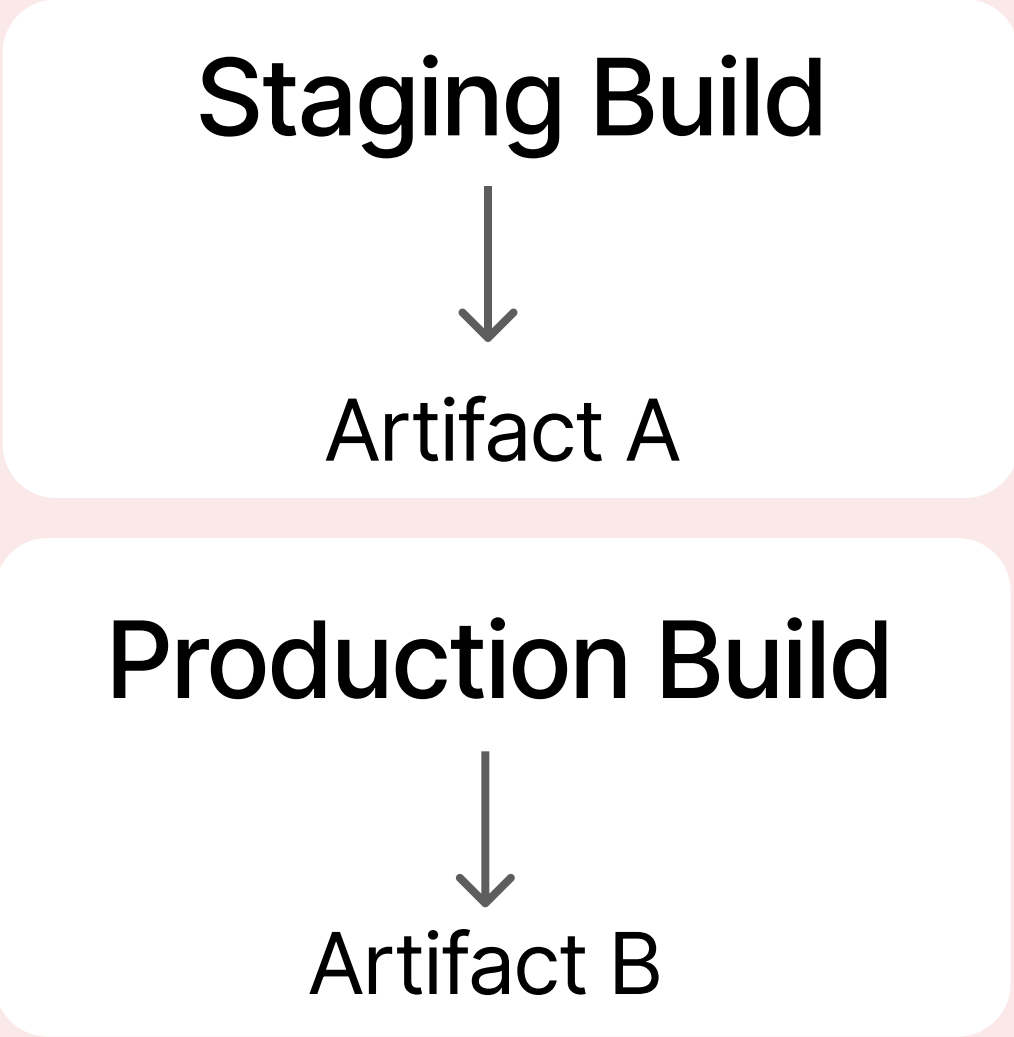
Container
Docker Image

Source → Build → Artifact

CI가 결과물을 만들고 검증하면, CD가 그 결과물을 전달한다

Build Once, Deploy Many

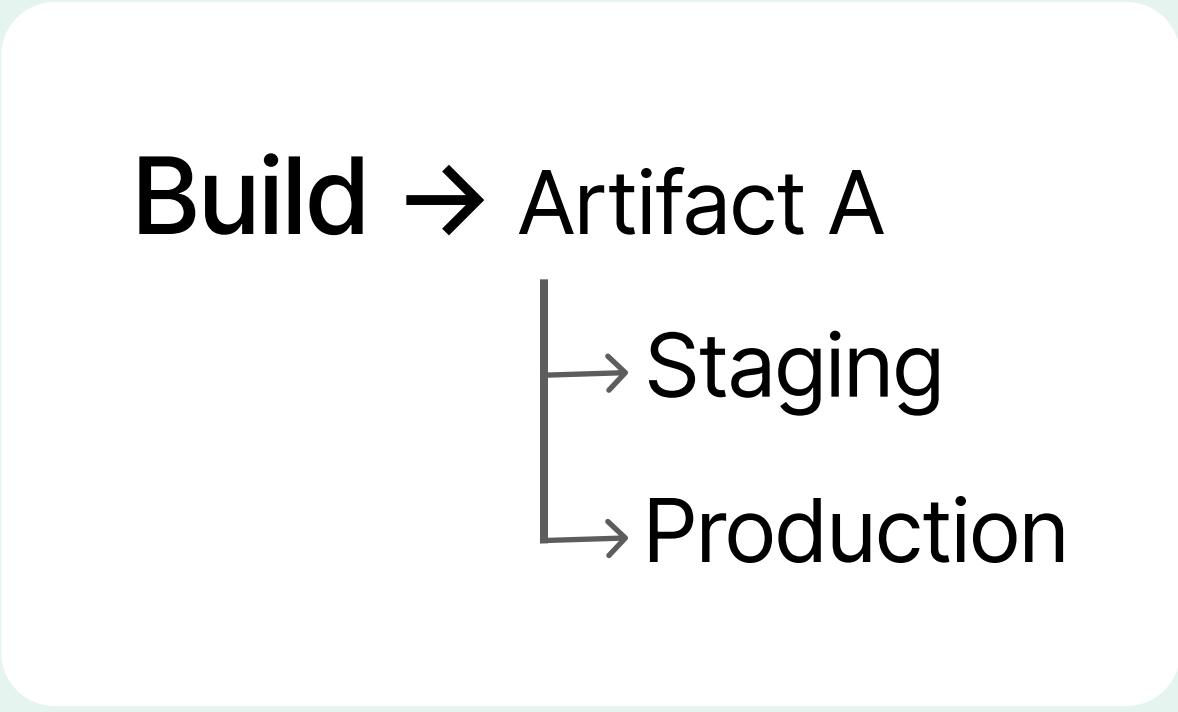
환경마다 다시 Build



결과가 달라질 수 있음

VS

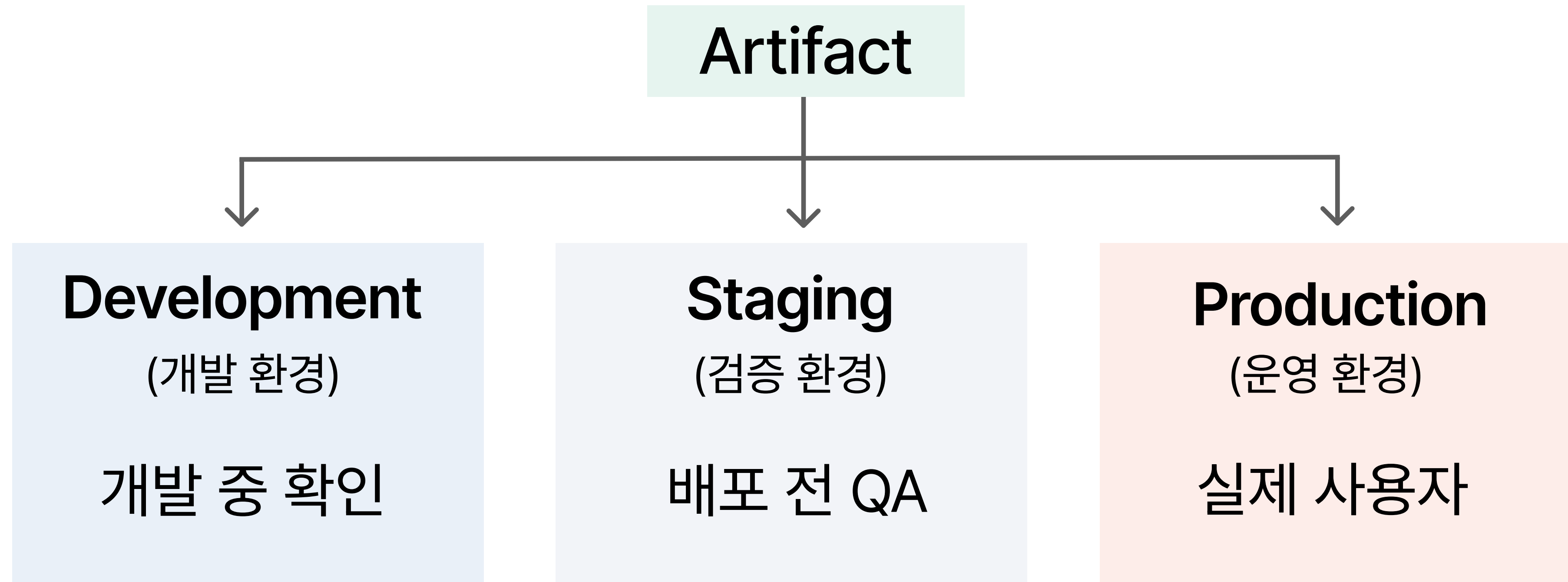
한 번 Build



검증한 결과물을 그대로 사용

한 번 검증한 Artifact를 모든 환경에 동일하게 배포한다

배포 환경



코드는 동일하게 유지하고, 환경별 설정만 분리한다

환경 변수와 Secret

Environment Variable
(일반적인 환경 설정)

- API URL
- MODE=production
- REGION=seoul

Secret

(공개되면 안 되는 값)

- DATABASE_PASSWORD
- ACCESS_TOKEN
- API_KEY

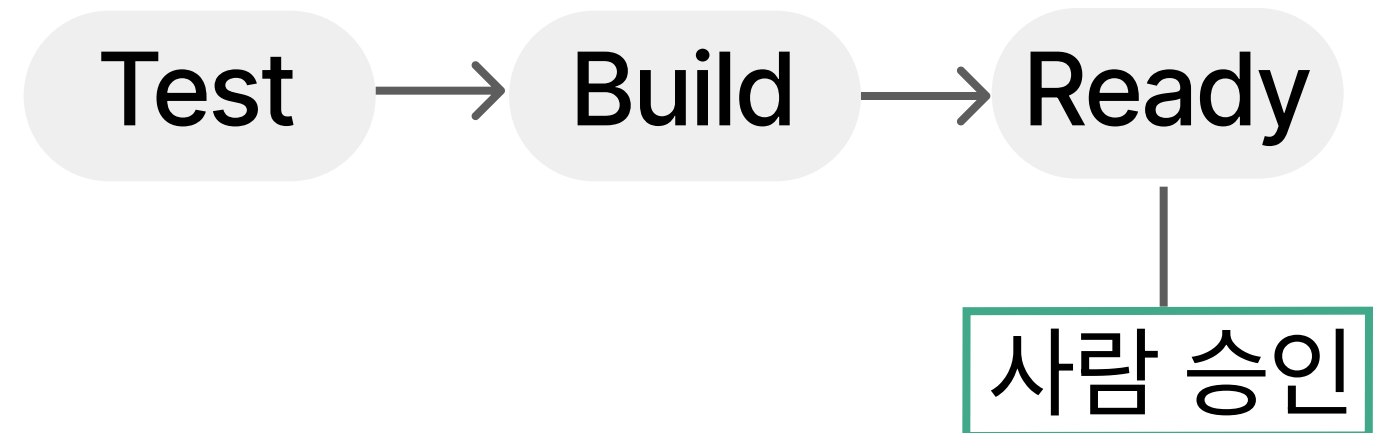


Application / Server

Secret은 코드·저장소·실행 로그에 노출하지 않는다.

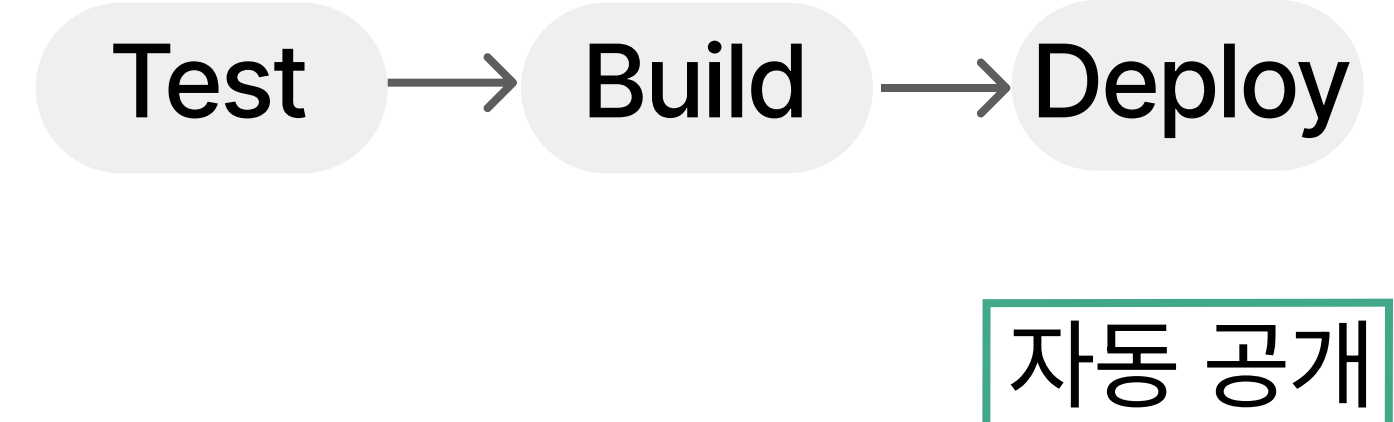
Continuous Delivery vs Deployment

Continuous Delivery (지속적 제공)



“배포 준비 완료”

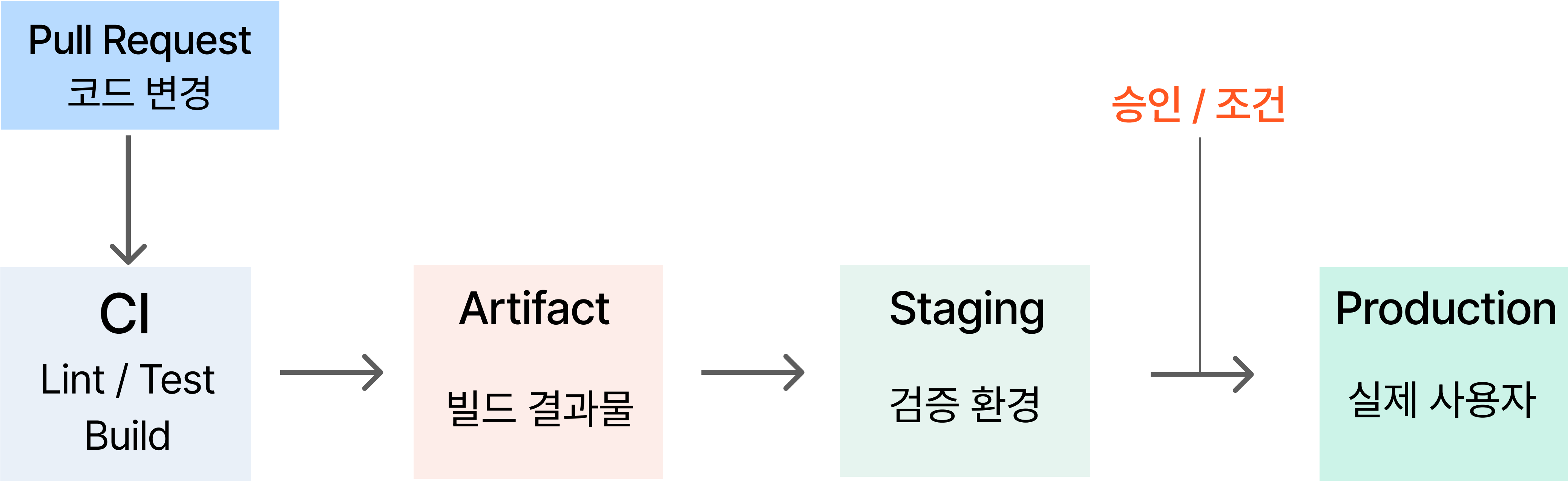
Continuous Deployment (지속적 배포)



“통과하면 바로 공개”

차이는 마지막 운영 배포를 누가 결정하는가이다

전체 파이프라인



Pull Request → CI → Artifact → Staging → 승인 → Production

오늘 기억할 점

CI
자동으로 검증



CD
안전하게 전달



목표
더 자주
더 안전하게
배포하기

자동화의 목적은 사람을 대신하는 것이 아니라 사람의 실수를 줄이는 것이다